

Text Chunking Strategies

What it is

This concept lives in `bilingual_etl/transform/chunker.py`. It takes the finished narrative produced by `narrative_builder.py` (see `docs/learning/09-narrative-template-design.md`) and splits it into pieces of **at most 500 words each** — this is **R7**. Each piece ("chunk") also gets a short header stamped on the front identifying which company and profile area it belongs to.

Like the narrative builder, this module makes no LLM calls and is pure Python: regex-based sentence splitting plus word counting. Its output is what actually gets stored, one row per chunk, in the `company_vectors` table — each row holds one chunk's text, its embedding vector, and its BM25 tokens.

Real-life analogy

Imagine you wrote a single, continuous 5,000-word essay covering everything about your company — history, products, services, certifications, and a giant list of every product category you touch — with no paragraph breaks, no headings, nothing. Now imagine asking a librarian to summarize "what this document is about" in one sentence. They'd have to average together the founding story, the product list, the certifications, and the 600-item category list into one mushy, vague summary — because you asked them to describe the *whole* document at once, and the whole document is about a dozen different things.

Now imagine the same content split into short, focused sections — "Company Background," "Products," "Certifications," "Categories We Serve" — each just a few hundred words. Ask the librarian to describe *one section*, and they can give you something sharp and specific: "this section lists the company's ISO certifications." That's the entire idea behind chunking: smaller, focused pieces of text produce sharper, more specific meaning than one giant piece that tries to be about everything at once.

Why we used it here

The reason this matters for search specifically: embedding a 5,000-word company profile as a *single* vector would blur every topic in it into one "average meaning" point. A user searching for one narrow thing — say, "ISO 9001 certified pump manufacturer" — wouldn't get a good match, because the company's single vector also has founding-year facts, a giant category list, and unrelated service descriptions all mixed into the same point in vector space. Splitting the narrative into focused, ≤500-word chunks means each chunk's embedding represents a narrower, more specific slice of the company's profile — which is what makes search results precise instead of mushy.

Sentence-aware splitting

The chunker doesn't just cut text every 500 words regardless of what's there — it prefers to split on sentence boundaries:

```
_SENTENCE_SPLIT_PATTERN = re.compile(r"(?<=[.!?])\s+")
```

```
def _split_sentences(text: str) -> list[str]:  
    return [s for s in _SENTENCE_SPLIT_PATTERN.split(text.strip()) if s.strip()]
```

The regex looks for a `.`, `!`, or `?` followed by whitespace, and splits there. `chunk_text()` then walks through the resulting sentence list, accumulating whole sentences into the current chunk until adding the *next* sentence would push it over the word limit — at which point the current chunk is closed off and a new one starts:

```
def chunk_text(text: str, max_words: int = MAX_WORDS_PER_CHUNK) -> list[str]:  
    if not text or not text.strip():  
        return []  
  
    sentences = _split_sentences(text)  
    chunks: list[str] = []  
    current_words: list[str] = []  
  
    for sentence in sentences:  
        sentence_words = sentence.split()  
  
        if current_words and len(current_words) + len(sentence_words) > max_words:  
            chunks.append(" ".join(current_words))  
            current_words = []  
  
        current_words.extend(sentence_words)  
    ...
```

Cutting a chunk off mid-sentence would leave it ending on a dangling half-thought — "the company also offers installation and" — which embeds poorly (the embedding model has no idea what was supposed to come next) and reads badly if the chunk is ever shown directly to a user as a search result snippet. Respecting sentence boundaries avoids both problems.

The hard-split fallback — and the real edge case that made it necessary

Sentence-aware splitting has one obvious hole: what if a *single sentence* is itself longer than 500 words? The regex has nothing to split on inside it — from its perspective, that's one unbreakable unit.

This isn't a hypothetical. The real dataset exposed it directly. One actual company, "**Zultzer Pumpen Kft.**", is associated with **625 categories**. `narrative_builder.py` renders the entire category list as one natural-language sentence: "A cég a következő kategóriákban aktív: [category], [category], ..., [category]." — commas everywhere, but only a single period at the very end. Measured against the real data, that one sentence alone runs to roughly **1,456 words**. Sentence-boundary splitting alone cannot touch it; regex-wise, it is one sentence, full stop (literally).

The fix is a `while` loop immediately after the sentence-accumulation logic, which force-splits at exactly `max_words` regardless of sentence structure:

```
# A single sentence longer than the limit (e.g. a giant comma-separated  
# category list with no internal punctuation) must still be split.  
while len(current_words) > max_words:  
    chunks.append(" ".join(current_words[:max_words]))  
    current_words = current_words[max_words:]
```

This only ever activates when `current_words` has grown past `max_words` — i.e., after a single oversized sentence got added — and it keeps slicing off exactly 500 words at a time until what's left fits.

Real, measured result from testing against the actual Zultzer Pumpen Kft. narrative (1,553 words total, built by `build_company_narrative()`): running it through `chunk_text()` produces **4 chunks**:

Chunk	Word count	How it was produced
0	97 words	A clean split on a real sentence boundary — this chunk holds the legal form, founding year, employee count, status, description, activities, services, certificates, brands, and county count; it ends right where the category-list sentence begins.
1	500 words	Hard split — cuts directly through the middle of the 1,456-word category-list sentence.
2	500 words	Hard split — continues through the same sentence.
3	456 words	The remainder of the category-list sentence.

Chunk 0 is proof the sentence-aware path works exactly as intended when sentences are a normal length. Chunks 1 through 3 are proof the hard-split fallback is doing real, necessary work — without it, that one 1,456-word sentence would have become (or broken) a single oversized chunk, or would have needed to be excluded from chunking altogether.

The chunk header — why every chunk carries its own identity

```
def _build_header(company_name: str, profile_line: str, language: str) -> str:
    if language == "hu":
        return f"[CÉG:] {company_name}\n[PROFIL:] {profile_line}"
    return f"[COMPANY:] {company_name}\n[PROFILE:] {profile_line}"
```

`company_vectors` stores **one chunk per database row** — not the whole company profile bundled together. That means when a search query matches, say, chunk #3 out of a company's 4 chunks, that row comes back to the caller *on its own*, with no automatic link back to "the rest of the profile" unless something in the chunk itself says what it belongs to. Without a header, chunk #3 in the Zultzer example above is just a wall of category names with no indication of which company they belong to at all.

The header re-anchors every chunk with its own identity, so a chunk can stand completely alone as a self-contained search result: `[CÉG:] Zultzer Pumpen Kft. / [PROFIL:] Ipari szivattyú- és kompresszorgyártó, karbantartó és forgalmazó vállalat.` at the top of every one of that company's chunks means even chunk #3, seen with zero other context, still identifies itself immediately.

One clarification worth being precise about: "bilingual" here does **not** mean a single header mixes both languages together. It means the *system* supports two header tag sets — [CÉG:]/[PROFIL:] for Hungarian, [COMPANY:]/[PROFILE:] for English — and each row uses exactly one of them, matching its own row's language. A Hungarian-language chunk always gets the Hungarian tag pair; an English chunk always gets the English pair. They never appear together in the same chunk.

```
def chunk_company_narrative(
    narrative: str,
    company_name: str,
    profile_line: str,
    language: str,
    max_words: int = MAX_WORDS_PER_CHUNK,
) -> list[str]:
    header = _build_header(company_name, profile_line, language)
    body_chunks = chunk_text(narrative, max_words=max_words)
    return [f"{header}\n\n{chunk}" for chunk in body_chunks]
```

This is also why the header is computed *once* and prepended to every resulting body chunk — every chunk from the same company/language pair gets an identical header.

Why the header doesn't count against the 500-word limit

Notice that `chunk_text()` — the function that actually enforces `max_words` — never sees the header at all; it only ever operates on the raw narrative text. The header gets stitched on afterward, in `chunk_company_narrative()`. Measured on the real Zultzer example: chunk 0's *body* is 97 words, but with the 13-word header prepended ([CÉG:] Zultzer Pumpen Kft. / [PROFIL:] Ipari szivattyú- és kompresszorgyártó, karbantartó és forgalmazó vállalat.), the full stored chunk is 110 words — over the notional "500" only in the loosest sense, and nowhere close to it in practice.

This is intentional. The header is small, fixed-size retrieval metadata — it exists to answer "whose chunk is this," not to be searchable prose content in its own right. If it were counted toward the 500-word limit, every chunk's real, meaningful content would be arbitrarily shrunk by a handful of words for no actual benefit — the embedding and BM25 value of the chunk comes from its body, not from re-stating the company name and profile line for the tenth time.

Synthetic questions go in *before* chunking, not into a separate field

```
def append_synthetic_questions(narrative: str, questions: list[str], language: str) -> str:
    if not questions:
        return narrative

    heading = "Gyakran ismételt kérdések:" if language == "hu" else "Frequently asked questions:"
    questions_block = " ".join(questions)
    return f"{narrative} {heading} {questions_block}"
```

The synthetic buyer questions generated by the AI enrichment step in the previous phase (e.g. "What pump models do you offer?", "Do your products come with a warranty?") get appended straight onto the end of the full narrative text, *before* `chunk_text()` ever runs on it. Confirmed on the real example above: appending two synthetic Hungarian questions to the 1,553-word Zultzer narrative produces a 1,564-word combined text (11 words for the heading plus the two questions), and that combined text is what gets chunked.

The design choice that matters here is the *order of operations*. Because the questions are appended before chunking, they land inside whichever chunk ends up covering that part of the text, and from that point on they are indistinguishable from any other sentence in the narrative — just as searchable, just as embeddable, just as likely to surface in a BM25 or vector search result. The alternative — storing synthetic questions in a separate, untouched database field — would make them effectively invisible to search, since neither the embedding pipeline nor the BM25 pipeline reads arbitrary side fields; they only ever process what actually ends up inside a chunk.

How to explain this in an interview/to a teammate

"A company profile can run to thousands of words, and embedding that whole thing as one vector would blur every topic in it — founding facts, certifications, a huge category list — into one vague 'average' meaning, which makes search results imprecise. So we chunk each narrative into pieces of at most 500 words, preferring to split on real sentence boundaries so we never cut a sentence in half. But real data broke that assumption almost immediately: one company in our dataset has 625 categories, and our narrative builder renders that as a single, giant, 1,400-plus-word sentence with commas but no internal periods — a regex-based sentence splitter can't touch it, because to a regex it's just one sentence. So we added a hard-split fallback that force-cuts at exactly 500 words whenever a single sentence is longer than the limit, and that's exactly what turns that one real company's profile into four chunks — a clean 97-word intro chunk, then three chunks that hard-split straight through the middle of the category list. We also stamp a small header on every chunk — company name and a short profile line — because our database stores one chunk per row, not the whole profile together, so without a header a chunk retrieved in isolation would have no idea what company it even belongs to. That header doesn't count toward the 500-word limit, since it's fixed retrieval metadata, not searchable content. And synthetic buyer questions from the enrichment step get appended to the narrative before chunking runs, specifically so they end up baked into a real, searchable chunk instead of sitting in some side field that search never actually looks at."